

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (Bsc.IT Semester III)
Data Structures Practical

Practical - VI

Roll No. :S037	Name:Himanshu Shrivastav
Class :SYIT	Batch : 00
Date of Assignment :29-07-2026	Date/Time of Submission :29-07-2026

PART – A : Thoery

A **Binary Tree** is a non-linear data structure in which each node has at most two children: a **left child** and a **right child**.

Tree Traversal is the process of visiting every node of a binary tree exactly once in a specific order. The three basic traversal methods are:

- **Pre-order Traversal (Root → Left → Right):** Visits the root node first, then the left subtree, and finally the right subtree.
- **In-order Traversal (Left → Root → Right):** Visits the left subtree first, then the root node, and finally the right subtree.
- **Post-order Traversal (Left → Right → Root):** Visits the left subtree first, then the right subtree, and finally the root node.

These traversal techniques are commonly used for searching, displaying, and processing the nodes of a binary tree. They are usually implemented using **recursion**.

PART – B : Question

Tree Traversal: Write a program to:

- a. Implement pre-order,
- b. in-order,
- c. Post-order traversal of a binary tree.

Code:

```
class Node:
```

```
    def __init__(self,item):
```

```
        self.left=None
```

```
self.right=None
```

```
self.val=item
```

```
def inorder(root):
```

```
    if root:
```

```
        inorder(root.left)
```

```
        print(str(root.val)+"->",end="")
```

```
        inorder(root.right)
```

```
def postorder(root):
```

```
    if root:
```

```
        postorder(root.left)
```

```
        postorder(root.right)
```

```
        print(str(root.val)+"->",end="")
```

```
def preorder(root):
```

```
    if root:
```

```
        print(str(root.val)+"->",end="")
```

```
        preorder(root.left)
```

```
        preorder(root.left)
```

```
root=Node(1)
```

```
root.left=Node(2)
```

```
root.right=Node(3)
```

```
root.left.left=Node(4)
```

```
root.left.right=Node(5)
```

```
print("Inorder Traversal")
```

```
inorder(root)
```

```
print("\nPostorder traversal")
```

```
postorder(root)
```

```
print("\nPreorder traversal")
preorder(root)
print("\nHimanshu S037")
```

Output:

```
Inorder Traversal
4->2->5->1->3->
Postorder traversal
4->5->2->3->1->
Preorder traversal
1->2->4->4->2->4->4->
Himanshu S037
```